



UNIT 4: FUNCTIONS LIST AND TUPLES

Prachi Surlakar
Assistant Professor
Computer Engineering Department
Goa College of Engineering

Unit - 4

Functions: Difference between function and method, defining, calling, returning result, returning multiple values from functions, formal and actual parameters, positional, keyword and default arguments, variable length arguments, local and global variables, passing a group of elements to a function.

List and Tuples: Creating lists using range () function, updating concatenating, repetition of lists, methods to process lists, finding the biggest and smallest element in a list, sorting the list elements, tuples, creating, accessing tuples, basic operations on tuples.

FUNCTIONS

- A function consists of a group of statements that are intended to perform a specific task.
- Built in () functions: print(), sqrt (), etc.
- **User defined functions:** Functions created by the programmer.
- **Advantages of functions:**
 - ❖ Functions are used to process data, make calculations, or perform any task which is required for software development.
 - ❖ Once function is written it can be reused as and when required.
 - ❖ Provide modularity for programming.
 - ❖ Code maintenance is easy.
 - ❖ When there is error corresponding function can be modified.
 - ❖ Use of functions will reduce length of a program.

DIFFERENCE BETWEEN FUNCTION AND A METHOD

- A **function** can be written individually in a python program.
- A **function** is called using its name.
- When a **function** is written inside a **class**, it becomes a **method**.
- A method is called using the following ways.

```
objectname.methodname()  
Classname.methodname()
```

- So a function and a method are same except their placement and the way they are called.

DEFINING A FUNCTION

- We can define a function using the **def** keyword followed by function name.
- After the function name we should write parenthesis()
- **Parameter:** is a variable that receives data from outside into a function.
- After parenthesis we put a colon(:) that represents the beginning of a function.
- Function body contains a group of statements called suite.
- Docstring : Optional. Gives information about the function.

Function definition:

```
def functionname( para1, para2, ...):  
    """ function docstring """  
    function statements
```

Example:

```
def sum(a, b):  
    """This function finds sum of two numbers"""  
    c=a+b  
    print(c)
```

CALLING A FUNCTION

- A function cannot run on its own. It runs only when we call it. We have to call the function using its name.

Write a function that finds the sum of two numbers

```
def sum(a, b):  
    """ This function finds sum of two numbers """  
    c = a + b  
    print('Sum =', c)  
  
# call the function  
sum(10, 15)  
sum(1.5, 10.75) # call second time
```

OUTPUT:

Sum = 25

Sum = 12.25

RETURNING RESULTS FROM A FUNCTION

- We can return the result or the output from the function using the return statement.

Write a python program to find the sum of two numbers and return the result from the function

```
# a function to add two numbers
def sum(a, b):
    """ This function finds sum of two numbers """
    c = a+b
    return c # return result
# call the function
x = sum(10, 15)
print('The sum is: ', x)
y = sum(1.5, 10.75)
print('The sum is: ', y)
```

OUTPUT:

```
The sum is: 25
The sum is: 12.25
```

Write a python program to check if a number is even or odd using functions.

```
# a function to test whether a number is even or odd
def even_odd(num):
    """ to know num is even or odd """
    if num % 2 == 0:
        print(num, " is even")
    else:
        print(num, " is odd")

# call the function
even_odd(12)
even_odd(13)
```

OUTPUT:

```
12 is even
13 is odd
```


RETURNING RESULTS FROM A FUNCTION

Write a python program to find factorial of numbers from 1 to 10 using functions

EXPLANATION

```
def factorial(n):  
    fact=1  
    i=n  
    while i>=1:  
        fact=fact*i  
        i-=1  
    return fact
```

Example:

n=3

fact =1

i=n =3

Iteration 1	Iteration 2	Iteration 3	Iteration 4
<u>fact</u> =1 <u>i</u> =3 3>=1 fact= 1*3=3 <u>i</u> =3-1=2	<u>i</u> =2 2>=1 fact= 3*2=6 <u>i</u> =2-1=1	<u>i</u> =1 1>=1 fact=6*1=6 <u>i</u> =1-1 =0	<u>i</u> =0 0>=1 STOP

Print fact value (6)

3!=6

PRACHI SURLAKAR

RETURNING RESULTS FROM A FUNCTION

Write a python program to find factorial of numbers from 1 to 10 using functions

```
def factorial(n):  
    fact=1  
    i=n  
    while i>=1:  
        fact=fact*i  
        i-=1  
    return fact  
for i in range(1,11):  
    print("factorial values",factorial(i))
```

OUTPUT

```
factorial values 1  
factorial values 2  
factorial values 6  
factorial values 24  
factorial values 120  
factorial values 720  
factorial values 5040  
factorial values 40320  
factorial values 362880  
factorial values 3628800
```


RETURNING RESULTS FROM A FUNCTION: Write a program using functions to check if a given number is prime or not

```
# a function to test whether a number is prime or not
def prime(n):
    """ to check if n is prime or not """
    x=1 # this will be 0 if not prime
    for i in range(2, n): # divide n from 2 to n-1
        if n%i == 0: # if divisible by any number
            x=0 # take x as 0
            break
    else:
        x=1 # else take x as 1
    return x
```

```
# test if a number is prime or not
num = int(input('Enter a number: '))

# check if num is prime or not
result = prime(num)
if result == 1:
    print(num, ' is prime')
else:
    print(num, ' is not prime')
```

Example 1:

n= 5

For i in range(2,5) – values 2,3,4

5%2==0

5%3==0

5%4==0

x=1

return 1

Print 5 is prime

Example 2:

n= 4

For i in range(2,4) – values 2,3

4%2==0

x=0 break

return 0

Print 4 is not prime

OUTPUT:

Enter a number: 5
5 is prime

Write a program to check if a given number is prime or not

```
# a function to test whether a number is prime or not
def prime(n):
    """ to check if n is prime or not """
    x=1 # this will be 0 if not prime
    for i in range(2, n): # divide n from 2 to n-1
        if n%i == 0: # if divisible by any number
            x=0 # take x as 0
            break
    else:
        x=1 # else take x as 1
    return x

# test if a number is prime or not
num = int(input('Enter a number: '))

# check if num is prime or not
result = prime(num)
if result == 1:
    print(num, ' is prime')
else:
    print(num, ' is not prime')
```

Example 1:

n= 5

For i in range(2,5) – values 2,3,4

5%2==0

5%3==0

5%4==0

x=1

return 1

Print 5 is prime

Example 2:

n= 4

For i in range(2,4) – values 2,3

4%2==0

x=0 break

return 0

Print 4 is not prime

OUTPUT:

Enter a number: 5
5 is prime

PRACHI SURLAKAR

RETURNING MULTIPLE VALUES FROM A FUNCTION

- Function can return multiple values in python.
- **Example:** return a, b, c
- Values are returned by a function as a tuple.

Write a python program to show how a function returns multiple values.

```
# a function that returns two results
def sum_sub(a, b):
    """ this function returns results of
        addition and subtraction of a, b """
    c = a + b
    d = a - b
    return c, d

# get the results from the sum_sub() function
x, y = sum_sub(10, 5)
# display the results
print("Result of addition: ", x)
print("Result of subtraction: ", y)
```

OUTPUT:

```
Result of addition: 15
Result of subtraction: 5
```

RETURNING MULTIPLE VALUES FROM A FUNCTION

Write a python function that returns multiple values of addition, subtraction, multiplication and division

CODE:

```
def sum_sub_mul_div(a, b):  
    c = a + b  
    d = a - b  
    e = a * b  
    f = a / b  
    return c, d, e, f
```

```
t = sum_sub_mul_div(10, 5)
```

```
# display the results using for loop  
print('The results are: ')  
for i in t:  
    print(i)
```

OUTPUT:

```
The results are:  
15  
5  
50  
2.0
```


FORMAL AND ACTUAL ARGUMENTS

Formal arguments: When a function is defined, it may have some parameters. These parameters are useful to receive values from outside the function.

Actual arguments: When we call the function we should pass data or values to the function. These values are called actual arguments.

```
def sum(a, b):  # a, b are formal arguments
    c = a + b
    print(c)
```

```
# call the function
```

```
x = 10
```

```
y = 15
```

```
sum(x, y)  # x, y are actual arguments
```

ACTUAL ARGUMENTS

The actual arguments are of 4 types:

1. **Positional arguments.**
2. Keyword arguments.
3. Default arguments.
4. Variable length arguments

Positional Arguments

- Number of arguments and their positions in the function definition should match exactly with the number and position of argument in the function call.

Python program to understand positional arguments

```
# positional arguments
def attach(s1, s2):
    s3 = s1 + s2
    print('Total string: ' + s3)
```

```
attach('New', 'York')
attach('York', 'New')
```

OUTPUT:

```
Total string: NewYork
Total string: YorkNew
```


ACTUAL ARGUMENTS

The actual arguments are of 4 types:

1. Positional arguments.
2. **Keyword arguments.**
3. Default arguments.
4. Variable length arguments

keyword Arguments

- Arguments that identify the parameters by their names.

Python program to understand keyword arguments

```
# key word arguments demo
def grocery(item, price):
    """ to display the given arguments """
    print('Item = %s' % item)
    print('Price = %.2f' % price)

# call grocery() and pass 2 arguments
grocery(item='Sugar', price=50.75) # keyword arguments
grocery(price=88.00, item='oil')  # keyword arguments
```

OUTPUT:

```
Item = Sugar
Price = 50.75
Item = oil
Price = 88.00
```

ACTUAL ARGUMENTS

The actual arguments are of 4 types:

1. Positional arguments.
2. Keyword arguments.
3. **Default arguments.**
4. Variable length arguments

Default Arguments

- Default argument: is an argument that assumes a default value if a value is not provided in the function call for that argument.

Python program to understand default arguments

```
# default arguments demo
def grocery(item, price=40.00):
    """ to display the given arguments """
    print('Item = %s' % item)
    print('Price = %.2f' % price)
# call grocery() and pass arguments
grocery(item='Sugar', price=50.75)    # pass 2 arguments
grocery(item='Sugar')                # default value for price is used.
```

OUTPUT:

```
Item = Sugar
Price = 50.75
Item = Sugar
Price = 40.00
```


ACTUAL ARGUMENTS

The actual arguments are of 4 types:

1. Positional arguments.
 2. Keyword arguments.
 3. Default arguments.
 4. **Variable length arguments**
- Sometimes programmer cannot decide how many arguments to be given in function definition.
 - If the programmer wants to develop a function that can accept n arguments then a **variable length argument** is used in function definition.
 - Variable length argument is written with a '*' symbol.

Variable length Arguments

Python program to show variable length argument and its use

```
def add(farg,*args):  
    print('formal argument=',farg)  
    sum=0  
    for i in args:  
        sum+=i  
    print("sum of all numbers",(farg+sum))
```

```
add(5,10)  
add(5,10,20,30)
```

OUTPUT:

```
formal argument= 5  
sum of all numbers 15  
formal argument= 5  
sum of all numbers 65
```

ACTUAL ARGUMENTS

The actual arguments are of 4 types:

1. Positional arguments.
 2. Keyword arguments.
 3. Default arguments.
 4. **Variable length arguments**
- Keyword variable length argument is an argument that can accept any number of values provided in the format of keys and values.
 - Declare it with a ' **' before the argument.

Variable length Arguments

Write a python program to understand keyword variable argument

```
def display(farg, **kwargs):  
    print('formal argument=', farg)  
    #items() method to get both keys and values  
    for x, y in kwargs.items():  
        print('key={}, value={}'.format(x, y))  
display(5, rollno=10)  
display(5, rollno=11, name='abc')
```

OUTPUT:

```
formal argument= 5  
key=rollno, value=10  
formal argument= 5  
key=rollno, value=11  
key=name, value=abc
```


LOCAL AND GLOBAL VARIABLES

- **Local variable:** declared inside a function. Scope is limited only to that function where it is created.

```
# local variable in a function
def myfunction():
    a=1    # this is local var
    a+=1   # increment it
    print(a)    # displays 2

myfunction()
print(a)    # error, not available
```

- **Global variable:** Scope of the global variable is the entire program body written below it.

```
# global variable example
a=1    # this is global var
def myfunction():
    b=2    # this is local var
    print('a= ', a)    # display globalvar
    print('b= ', b)    # display localvar
```

```
myfunction()
print(a)    # available
print(b)    # error, not available
```

When local and global variables have the same name, function by default refers the local variable.

PASSING A GROUP OF ELEMENTS TO A FUNCTION

To pass a group of elements like numbers or strings, we can accept them into a list and then pass the list to the function where the required processing can be done.

Write a function to accept a group of numbers and find their total and average

```
# a function to find total and average
def calculate(lst):
    """ to find total and average """
    n = len(lst)
    sum = 0
    for i in lst:
        sum += i
    avg = sum / n
    return sum, avg

# take a group of integers from keyboard
print('Enter numbers separated by space: ')
lst = [int(x) for x in input().split()]

# call calculate() and pass the list
x, y = calculate(lst)
print('Total: ', x)
print('Average: ', y)
```

OUTPUT

```
Enter numbers separated by space:
10 20 30 40
Total: 100
Average: 25.0
```


PASSING A GROUP OF ELEMENTS TO A FUNCTION

Write a function to display a group of strings

```
# a function to display a group of strings
def display(lst):
    for i in lst:
        print(i)

print('Enter strings separated by comma: ')
lst = [x for x in input().split(',')]

display(lst)
```

OUTPUT:

```
Enter strings separated by comma:
abc,xyz,lmn,pqr
abc
xyz
lmn
pqr
```

Unit - 4

Functions: Difference between function and method, defining, calling, returning result, returning multiple values from functions, formal and actual parameters, positional, keyword and default arguments, variable length arguments, local and global variables, passing a group of elements to a function.

List and Tuples: Creating lists using range () function, updating concatenating, repetition of lists, methods to process lists, finding the biggest and smallest element in a list, sorting the list elements, tuples, creating, accessing tuples, basic operations on tuples.

LIST

- List can store a group of elements.
- Can store different types of elements.
- More versatile and useful than array.

To create an empty list

```
e_lst = [] # this an empty list
```

Create a list to store student information

```
student = [10, 'venu gopal', 'M', 50, 55, 62, 74, 66]
```

Print student list

```
print(student)
```

Print student name

```
print(student[1])
```

Slicing operations on lists:

Format: [start : stop : step size]

By default start will be zero and stop will be the last element and step size will be 1

```
student = [10, 'venu gopal', 'M', 50, 55, 62, 74, 66]
```

CODE:

```
print(student[0:3:1])
```

```
print(student[:3:])
```

```
print(student[::])
```

OUTPUT:

```
[10, 'venu gopal', 'M']
```

```
[10, 'venu gopal', 'M']
```

```
[10, 'venu gopal', 'M', 50, 55, 62, 74, 66]
```


LIST

Write a python program to create a list with different types of elements.

```
num=[10,20,30,40,50]
print('total list=',num)
print('first and last=',num[0],num[4])

names=["abc","xyz","pqr","lmn"]
print('total list=',names)
print('first and last=',names[0],names[3])

x=["abc","xyz","10","20.5"]
print('total list=',x)
print('first and last=',x[0],x[3])
```

OUTPUT:

```
total list= [10, 20, 30, 40, 50]
first and last= 10 50
total list= ['abc', 'xyz', 'pqr', 'lmn']
first and last= abc lmn
total list= ['abc', 'xyz', '10', '20.5']
first and last= abc 20.5
```

Creating lists using range() function

Write a python program to create following lists using range function

- List of numbers from 0 to 4
- List of numbers from 2 to 4
- Odd numbers from 5 to 10

OUTPUT:

```
list1=range(5)
for i in list1:
    print(i)
print()
```

0
1
2
3
4

```
list2=range(2,5)
for i in list2:
    print(i)
print()
```

2
3
4

```
list3=range(5,10,2)
for i in list3:
    print(i)
print()
```

5
7
9

Write a python program to access elements of a list using while and for loops.

```
# displaying list elements using while and for loops
list = [10,20,30,40,50]

print('Using while loop')
i=0
while i<len(list): # repeat from 0 to length of list
    print(list[i])
    i=i+1

print('Using for loop')
for i in list: # repeat for all elements
    print(i)
```

OUTPUT:

```
Using while loop
10
20
30
40
50
Using for loop
10
20
30
40
50
```

PRACHI SURLAKAR

UPDATING ELEMENTS OF A LIST

Lists are mutable. We can modify the contents of a list.

Create a list

```
lst = list(range(1,5))  
print(lst)
```

```
[1, 2, 3, 4]
```

Append elements to a list

```
lst.append(9)  
print(lst)
```

```
[1, 2, 3, 4, 9]
```

Update second element of a list

```
lst[1]= 8  
print(lst)
```

```
[1, 8, 3, 4, 9]
```


UPDATING ELEMENTS OF A LIST

Update second and third element

```
lst[1:3] = 10, 11  
print(lst)
```

OUTPUT:

```
[1, 10, 11, 4, 9]
```

Delete elements

```
del lst[1]  
print(lst)
```

```
[1, 11, 4, 9]
```

```
lst.remove(11)  
print(lst)
```

```
[1, 4, 9]
```

Reverse elements of a list

```
list.reverse()
```

Perform the following operations on lists

- **Create a list with elements from 0 to 9**
- **Append value 10 to the list**
- **Update the fourth element of the list to 40.**
- **Update the second and third element of a list to 20 and 30 respectively.**
- **Delete first element of the list.**
- **Reverse the list elements.**

UPDATING ELEMENTS OF A LIST

Write a python program to display elements of a list in reverse order

```
# displaying list elements in reverse order
days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday']

print('\nIn reverse order: ')
i=len(days)-1 # i will be 4
while i>=0:
    print(days[i]) # display from 4th to 0th elements
    i-=1

print('\nIn reverse order: ')
i=-1 # days[-1] represents last element
while i>=-len(days): # display from -1th to -5th elements
    print(days[i])
    i-=1
```

OUTPUT

```
In reverse order:
Thursday
Wednesday
Tuesday
Monday
Sunday
```

```
In reverse order:
Thursday
Wednesday
Tuesday
Monday
Sunday
```

PRACHI SURLAKAR

CONCATENATION OF TWO LISTS

+ operator can be used on two lists to join them.

```
x = [10, 20, 30, 40, 50]  
y = [100, 110, 120]  
print(x+y)    # concatenate x and y
```

OUTPUT:

```
[10, 20, 30, 40, 50, 100, 110, 120]
```


REPETITION OF LISTS

We can repeat elements of a list 'n' number of times using the '*' operator.

```
x = [10,20,30,40,50]  
print(x*2)    # repeat the list x for 2 times
```

OUTPUT:

```
[10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
```


METHODS TO PROCESS LISTS

Method	Example	Description
sum()	list.sum()	Returns sum of all elements in the list.
index()	list.index(x)	Returns the first occurrence of x in the list
append()	list.append(x)	Appends x at the end of the list
insert()	list.insert(i, x)	Inserts x in to the list in the position specified by i
copy()	list.copy()	Copies all the list elements into a new list and returns it
extend()	list.extend(list1)	Appends list1 to list
count()	list.count(x)	Returns number of occurrences of x in the list
remove()	list.remove(x)	Removes x from the list
pop()	list.pop()	Removes the ending element from the list
sort()	list.sort()	Sorts the elements of the list into ascending order
reverse()	list.reverse()	Reverses the sequence of elements in the list
clear()	list.clear()	Deletes all elements from the list

PRACHI SURLAKAR

Write a python program to understand the list processing methods

```
# Python's list methods
num = [10,20,30,40,50]

n = len(num)
print('No. of elements in num: ', n)

num.append(60)
print('num after appending 60: ', num)

num.insert(0,5)
print('num after inserting 5 at 0th position: ', num)

num1 = num.copy()
print('Newly created list num1: ', num1)

num.extend(num1)
print('num after appending num1: ', num)

n = num.count(50)
print('No. of times 50 found in the list num: ', n)

num.remove(50)
print('num after removing 50: ', num)

num.pop()
print('num after removing ending element: ', num)

num.sort()
print('num after sorting: ', num)

num.reverse()
print('num after reversing: ', num)

num.clear()
print('num after removing all elements: ', num)
```

Write a python program to understand the list processing methods

OUTPUT:

```
Python Lists.py
No. of elements in num: 5
num after appending 60: [10, 20, 30, 40, 50, 60]
num after inserting 5 at 0th position: [5, 10, 20, 30, 40, 50, 60]
Newly created list num1: [5, 10, 20, 30, 40, 50, 60]
num after appending num1: [5, 10, 20, 30, 40, 50, 60, 5, 10, 20, 30,
40, 50, 60]
No. of times 50 found in the list num: 2
num after removing 50: [5, 10, 20, 30, 40, 60, 5, 10, 20, 30, 40, 50, 60]
num after removing ending element: [5, 10, 20, 30, 40, 60, 5, 10, 20,
30, 40, 50]
num after sorting: [5, 5, 10, 10, 20, 20, 30, 30, 40, 40, 50, 60]
num after reversing: [60, 50, 40, 40, 30, 30, 20, 20, 10, 10, 5, 5]
num after removing all elements: []
```


FINDING BIGGEST AND SMALLEST ELEMENTS IN A LIST

`max()` and `min()` functions return the biggest and the smallest element from the list.

```
x = [10, 20, 30, 40, 50]  
print(max(x))  
print(min(x))
```

Consider user inserts the following values using for loop

a[0]	a[1]	a[2]	a[3]	a[4]
10	30	20	40	5

large=small=a[0]=10 (first element)

2 formulas:

if(a[i]>large) large=a[i]	if(a[i]<small) small=a[i]
------------------------------	------------------------------

a[0]>large 10>10 no	a[0]<small 10<10 no	large=10 small=10
a[1]>large 30>10 Yes large =30	a[1]<small 30<10 no	large=30 small=10
a[2]>large 20>30 no	a[2]<small 20<10 no	large=30 small=10
a[3]>large 40>30 Yes large=40	a[3]<small 40<10 no	large=40 small=10
a[4]>large 5>40 no	a[5]<small 5<10 Yes small=5	large=40 small=5

Print: largest=40 smallest =5

FINDING BIGGEST AND SMALLEST ELEMENTS IN A LIST without using max() and min() functions.

FINDING BIGGEST AND SMALLEST ELEMENTS IN A LIST without using max() and min() functions.

```
#finding biggest and smallest numbers in a list of numbers
```

```
x = [] # take an empty list
```

```
print('How many elements? ', end='')
```

```
n = int(input()) # accept input into n
```

```
for i in range(n): # repeat for n times
```

```
    print('Enter element: ', end='')
```

```
    x.append(int(input()))
```

```
print('The list is: ', x) # display the list
```

```
large = x[0] # initially 0th element becomes maximum
```

```
small = x[0] # initially 0th element becomes minimum
```

```
for i in range(0, n):#repeat from 0 to n-1 elements
```

```
    if x[i] > large:
```

```
        large = x[i]
```

```
    if x[i] < small:
```

```
        small = x[i]
```

```
print('Largest element is:', large)
```

```
print('Smallest element is:', small)
```

How many elements? 5

Enter element: 10

Enter element: 20

Enter element: 3

Enter element: 90

Enter element: 2

The list is: [10, 20, 3, 90, 2]

Largest element is: 90

Smallest element is: 2

SORTING THE LIST ELEMENTS

`sort()` method: sorts elements of a list.

Example:

`x.sort()` # sorts elements in list x in sorted order

`x.sort(reverse=True)` # sorts list x in descending order

SORTING WITHOUT USING THE SORT() METHOD

Original list: 1, 5, 4, 3, 2

Compare 1 with other elements. 1 is smallest hence no swaps. we get:

1, 5, 4, 3, 2

Compare 5 with other elements. Swap 5 with 4, 5 with 3, 5 with 2. we get: 1, 4, 3, 2, 5

Compare 4 with other elements. Swap 4 with 3, 4 with 2. we get: 1, 3, 2, 4, 5

Compare 3 with other elements. Swap 3 with 2. we get: 1, 2, 3, 4, 5

SWAPPING

```
t = a
a = b
b = t
```



```

x = []
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n): # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input())) # add the element to the list x

print('Original list: ', x)
# bubble sort
flag = False # when swapping is done, flag becomes True
for i in range(n-1): # i is from 0 to n-1
    for j in range(n-1-i): # j is from 0 to one element lesser than i
        if x[j] > x[j+1]: # if 1st element is bigger than the 2nd one
            t = x[j] # swap j and j+1 elements
            x[j] = x[j+1]
            x[j+1] = t
            flag = True # swapping done, hence
        if flag == False: # no swapping means list
            break # come out of inner for loop
    else:
        flag = False # assign initial value to
print('Sorted list: ', x)

```

OUTPUT

How many elements? 5

Enter element: 10

Enter element: 3

Enter element: 20

Enter element: 4

Enter element: 90

Original list: [10, 3, 20, 4, 90]

Sorted list: [3, 4, 10, 20, 90]

PRACHI SURLAKAR

TUPLE

- Tuple stores a group of elements or items.
- Tuples are immutable whereas lists are mutable.
- We cannot perform operations like `append()`, `insert()`, `remove()`, `pop()` and `clear()` on tuples.
- Tuples used to store data which should not be modified.
- We can create a tuple by writing elements inside parentheses ()

Different ways of creating tuples

```
tup1=()
tup2=(10,) # without comma interpret as a integer
tup3 = (10, 20, -30.1, 40.5, 'Hyderabad', 'New Delhi')
tup4 = (10, 20, 30) # tuple with integers
tup5=1,2,3,4 # if no brackets then by default it is taken as a tuple
```


TUPLE

Different ways of creating tuples

```
#create tuple from a list  
lst=[1,2,3]  
tp1=tuple(lst)  
print(tp1)
```

```
#create tuple using range function  
tp1=tuple(range(4,9,2))  
print(tp1)
```

OUTPUT:

```
(1, 2, 3)  
(4, 6, 8)
```

ACCESSING THE TUPLE ELEMENTS

```
tup = (50,60,70,80,90,100)
```

```
print(tup[0])
```

50

```
print(tup [5])
```

100

```
print(tup[-1])
```

100

```
print(tup[-6])
```

50

SLICING OPERATIONS ON TUPLES

[start: stop: stepsize]

```
tup = (50,60,70,80,90,100)
```

```
print(tup[:])
```

```
(50, 60, 70, 80, 90, 100)
```

```
print(tup[1:4])
```

```
(60, 70, 80)
```

```
print(tup[::2])
```

```
(50, 70, 90)
```

```
print(tup[::-2])
```

```
(100, 80, 60)
```

```
print(tup[-4:-1])
```

```
(70, 80, 90)
```

BASIC OPERATIONS ON TUPLES

Len (): finds the length of a tuple

```
student = (10, 'vinaykumar', 50,60,65,61,70)
len(student)
```

Concatenate using the + operator

```
student = (10, 'vinaykumar', 50,60,65,61,70)
```

```
fees = (25000.00,)*4
```

```
student1 = student+fees
print(student1)
```

OUTPUT:

```
(10, 'vinay kumar', 50, 60, 65, 61, 70, 25000.0, 25000.0, 25000.0,
25000.0)
```

'in' and 'not in' operator

```
name='vinay kumar'
name in student1
```

True

```
name not in student1
```

False

REPITITION OPERATOR

```
tpl = (10, 11, 12)
tpl1 = tpl*3    # repeat for 3 times and store in tpl1
print(tpl1)
```

OUTPUT:

```
(10, 11, 12, 10, 11, 12, 10, 11, 12)
```

DIFFERENTIATE BETWEEN LISTS AND TUPLES

Feature	List	Tuple
Mutability	Mutable (can be modified after creation)	Immutable (cannot be modified after creation)
Syntax	Defined using square brackets <code>[]</code>	Defined using parentheses <code>()</code>
Methods	Many methods like <code>append()</code> , <code>remove()</code> , <code>pop()</code>	Limited methods like <code>count()</code> and <code>index()</code>
Performance	Slightly slower due to mutability and higher overhead	Faster due to immutability and optimized memory usage
Memory Usage	Requires more memory	More memory-efficient

MORE PATTERNS IN PYTHON

```
n = int(input("Enter the size of the square: "))
# Loop through each row
for i in range(n):
    if i == 0 or i == n - 1:
        # Print the full row for the first and last rows
        print("* " * n)
    else:
        # Print the hollow row with spaces in between
        print("* " + " " * (n - 2) + "* ")
```

Enter the size of the square: 5

```
* * * * *
*         *
*         *
*         *
*         *
* * * * *
```

```
for i in range(1, 6):  
    for j in range(1, i + 1):  
        print(j, end=" ")  
    print()
```

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```



```
for i in range(5):  
    for j in range(i + 1):  
        # Print each character without a newline at the end  
        print(chr(65 + j), end=" ")  
    # Print a newline after each row  
    print()
```

```
A  
A B  
A B C  
A B C D  
A B C D E
```

THANK YOU😊

by PRACHI SURLAKAR

PRACHI SURLAKAR